Phase 2 OOP Implementation

Project Name: Roommate Dashboard

Document ID: Phase2_OOP_Implementation

Control Label: D0-v1-r1

Date: 4/12/2026

Prepared By: Group 6

1. Project Overview & Where Everything Is

Roommate Dashboard is a Spring Boot web application that helps roommates manage shared household responsibilities including expenses, chore assignments, grocery lists, and payment tracking. It follows MVC architecture with JPA persistence and Thymeleaf server-rendered templates.

1.1 Repository & File Structure

All source code resides under src/main/java/com/tammam/roomatedash/ organized into five packages:

Package	Purpose	Key Files
model/	JPA entity classes (domain objects)	AppUser, Household, Roommate, Expense, Chore, GroceryItem, Payment, HouseholdMember, ActivityLog, ExpenseSplit
service/	Business logic layer	HouseholdService, ExpenseService, ChoreService, GroceryService, BalanceService, PaymentService, DashboardSummaryService, ActivityLogService, RoommateService
controller/	MVC controllers & model builders	DashboardController, HouseholdController, AuthController, ChoreController, ExpenseSplitController, GroceryController, DashboardModelBuilder
repo/	Spring Data JPA repository interfaces	HouseholdRepository, ExpenseRepository, ChoreRepository, GroceryItemRepository, PaymentRepository, RoommateRepository, AppUserRepository, ExpenseSplitRepository, HouseholdMemberRepository, ActivityLogRepository
dto/	Form-binding data transfer objects	ExpenseForm, ChoreForm, GroceryItemForm, PaymentForm, HouseholdForm, JoinHouseholdForm, RegisterForm, LoginForm, DashboardSummary

1.2 Supporting Files

Location	Description
src/main/resources/templates/	Thymeleaf HTML templates: dashboard.html, login.html, register.html, household-new.html, household-join.html, error.html
src/main/resources/static/	dashboard.css — custom CSS for the dashboard UI
src/main/resources/application.properties	H2 in-memory database configuration for development
src/main/resources/application-postgres.properties	PostgreSQL profile for production deployment
data/roommate_demo_db.mv.db	Persistent H2 demo database with seed data
src/test/java/.../DashboardWorkflowTests.java	Integration tests covering core dashboard workflows
pom.xml	Maven build file — Spring Boot 3.x, Jakarta EE, Spring Data JPA, Thymeleaf, H2/PostgreSQL

2. Step One — Identifying Analysis Classes

The Identifying Analysis Classes method was applied by examining nouns from the system's domain narrative (roommates sharing a household, splitting bills, assigning chores) and filtering for candidates that carry state and behavior relevant to the SRS. Boundary, control, and entity stereotypes were assigned.

2.1 Entity Classes (Domain Objects)

Class	Stereotype	Role in Domain	Key Attributes
AppUser	Entity	Authenticated user; can belong to multiple households	id, name, email, createdAt
Household	Entity	Central grouping container — a home that roommates share	id, name, inviteCode, createdBy, createdAt

Class	Stereotype	Role in Domain	Key Attributes
HouseholdMember	Entity	Association between AppUser and Household; carries the role	id, household, user, role (ORGANIZER/MEMBER), joinedAt
Roommate	Entity	Named participant in a household who can be assigned expenses/chores	id, name, household
Expense	Entity	Shared bill with amount, payer, and split configuration	id, description, amount, dueDate, splitType, recurring, payer, household
ExpenseSplit	Entity	One roommate's share of an Expense; tracks paid status	id, expense, roommate, share, percentage, paid, paidAt
Payment	Entity	Direct money transfer logged between two roommates	id, fromRoommate, toRoommate, amount, note, household, createdAt
Chore	Entity	Household task assigned to a roommate with optional rotation	id, title, description, assignedTo, household, status, dueDate, rotating, completedAt
GroceryItem	Entity	Item on the shared grocery list	id, name, quantity, assignedTo, household, status, createdAt, purchasedAt
ActivityLog	Entity	Immutable audit record of actions taken in a household	id, household, actor, actionType, message, createdAt

2.2 Enumeration Classes

Enum	Values	Used By
HouseholdRole	ORGANIZER, MEMBER	HouseholdMember — determines permissions within a household
ChoreStatus	OPEN, COMPLETED	Chore — tracks whether a chore has been completed

Enum	Values	Used By
GroceryItemStatus	NEEDED, PURCHASED	GroceryItem — tracks whether an item has been bought

2.3 Control & Service Classes

Class	Stereotype	Responsibilities
HouseholdService	Control	Creates/joins households, manages membership, resolves current household context, ensures default household exists
ExpenseService	Control	Adds expenses with equal or custom splits, marks splits paid, generates next recurring bill
ChoreService	Control	Assigns chores, marks complete, rotates weekly rotating chores
GroceryService	Control	Adds grocery items, marks purchased
PaymentService	Control	Logs direct roommate-to-roommate payments
BalanceService	Control	Computes net balance summaries per roommate from expense splits and payments
RoommateService	Control	CRUD for Roommate entities scoped to a household
DashboardSummaryService	Control	Aggregates summary statistics (open chores, unpaid splits, recent activity) for the dashboard view
ActivityLogService	Control	Appends immutable log entries for all domain events
MockAuthService	Control	Session-based mock authentication — stores current user/household IDs in HttpSession
HouseholdContext	Control	Reads current household ID from the session for scoping queries

2.4 Boundary Classes

Class	Stereotype	Responsibilities
DashboardController	Boundary	Renders main dashboard; delegates to DashboardModelBuilder
HouseholdController	Boundary	Handles household creation, joining, and switching via form submissions
AuthController	Boundary	Handles user registration and login (mock, session-based)
ChoreController	Boundary	POST endpoints for adding, completing, and rotating chores
ExpenseSplitController	Boundary	POST endpoints for adding expenses, marking splits paid, generating recurring bills
GroceryController	Boundary	POST endpoints for adding grocery items and marking purchased
DashboardModelBuilder	Boundary	Helper that builds the Thymeleaf model map for the dashboard template
WebConfig	Boundary	Registers the MockAuthInterceptor so every request resolves the current user
MockAuthInterceptor	Boundary	Pre-handles every request to load session user/household into request attributes
GlobalExceptionHandler	Boundary	ControllerAdvice that catches DashboardException and ResourceNotFoundException; renders error page

2.5 DTO / Form Classes

Class	Used For
ExpenseForm	Binding expense add form — description, amount, payerId, splitRoommateIds, splitType, custom percentages, dueDate, recurring
ChoreForm	Binding chore add form — title, description, assignedToId, dueDate, rotating flag
GroceryItemForm	Binding grocery item add form — name, quantity, assignedToId
PaymentForm	Binding payment log form — fromRoommateId, toRoommateId, amount, note
HouseholdForm	Binding household creation form — name

Class	Used For
JoinHouseholdForm	Binding join household form — inviteCode
RegisterForm	Binding registration form — name, email
LoginForm	Binding login form — email
DashboardSummary	Read-only DTO carrying aggregate stats: openChoreCount, unpaidSplitCount, recentActivity list

3. Step Two — OOP Implementation

3.1 Encapsulation

All entity and DTO fields are declared private. State is accessed exclusively through public getters and setters. No field is exposed directly. Business logic that mutates state is encapsulated within domain methods rather than being performed externally.

Examples of encapsulated domain methods:

- Chore.complete() — atomically sets status = COMPLETED and records completedAt = now()
- GroceryItem.markPurchased() — atomically sets status = PURCHASED and records purchasedAt = now()
- ExpenseSplit.markPaid() — atomically sets paid = true and paidAt = now()
- Expense.getFormattedCreatedAt() / getFormattedDueDate() — formatting logic lives inside the entity, not in controllers or templates

3.2 Abstraction

The repository layer uses Spring Data JPA interfaces. All ten repositories extend JpaRepository<T, Long>, exposing a uniform contract (save, findById, findAll, delete) without any implementation code. Custom queries are declared as method signatures and resolved automatically.

Service classes abstract domain operations from controllers. Controllers never directly touch the database — they call service methods with high-level intent:

- expenseService.addExpense(form) — controller does not know about split calculation, RoundingMode, or which repositories are called
- choreService.rotateWeeklyChores() — controller does not know how rotation index is computed
- householdService.ensureOrCreateDefaultHousehold(session) — complex fallback logic is fully abstracted

DashboardModelBuilder further abstracts the model-population concern out of DashboardController, making the controller a thin orchestrator.

3.3 Inheritance

The exception hierarchy uses inheritance to separate concerns:

Class	Extends	Purpose
DashboardException	RuntimeException	Business-rule violation (e.g., percentages do not total 100, payer not found). Results in a user-visible error page.
ResourceNotFoundException	RuntimeException	404-style error when a requested entity does not exist in the database.
GlobalExceptionHandler	N/A (@ControllerAdvice)	Catches both exception types polymorphically and routes to the error template.

All JPA entity classes implicitly inherit the managed-entity lifecycle from the JPA specification through the @Entity annotation and Hibernate. All repository interfaces inherit full CRUD from JpaRepository<T, Long>.

3.4 Polymorphism

Polymorphism is expressed in several ways throughout the codebase:

Interface Polymorphism — Repositories

All ten repository interfaces share the JpaRepository<T, Long> contract. Service classes receive repositories through constructor injection typed to the interface — the concrete Hibernate proxy is swapped in at runtime without altering service code.

Exception Polymorphism — GlobalExceptionHandler

Two @ExceptionHandler methods each catch their own exception subtype (DashboardException, ResourceNotFoundException) and format the error page accordingly, demonstrating method-level polymorphic dispatch.

Behavioral Polymorphism — Status Enums

Controller and template logic branches on enum values (ChoreStatus.OPEN vs COMPLETED, GroceryItemStatus.NEEDED vs PURCHASED) enabling polymorphic

rendering: an OPEN chore renders a Complete button; a COMPLETED chore renders a completion timestamp.

Spring IoC Polymorphism — Constructor Injection

Service classes depend on interfaces injected by Spring's IoC container. In tests, mock implementations are substituted at the interface boundary, confirming runtime polymorphic substitution.

4. Step Three — SOLID Design Principles

4.1 Single Responsibility Principle (SRP)

Each class has one reason to change:

- ExpenseService — only responsible for expense and split business rules
- ChoreService — only responsible for chore assignment and rotation logic
- ActivityLogService — only responsible for writing audit log entries
- BalanceService — only responsible for computing net roommate balances
- DashboardModelBuilder — only responsible for assembling the Thymeleaf model map
- GlobalExceptionHandler — only responsible for mapping exceptions to error pages

4.2 Open/Closed Principle (OCP)

The system is open for extension and closed for modification:

- Adding a new expense split type (e.g., WEIGHTED) requires adding a new branch in ExpenseService.addExpense() without modifying the Expense entity or any controller
- Adding a new household action type in ActivityLogService requires no changes to any calling service
- New repository query methods are added to the JpaRepository interface without modifying existing methods

4.3 Liskov Substitution Principle (LSP)

DashboardException and ResourceNotFoundException both extend RuntimeException and are handled by GlobalExceptionHandler without any instanceof checks — each is substitutable for RuntimeException. All repository beans implement their declared JpaRepository interface and can be replaced by mocks in tests.

4.4 Interface Segregation Principle (ISP)

Each repository interface is specific to one entity type (ExpenseRepository, ChoreRepository, etc.) rather than a monolithic DAO. Service classes only inject the repositories they actually use. For example, ChoreService injects only ChoreRepository and related collaborators — it has no dependency on ExpenseRepository.

4.5 Dependency Inversion Principle (DIP)

All service and controller classes receive dependencies through constructor injection typed to interfaces or Spring-managed beans. No class instantiates its own collaborators with new. The @Service and @Repository annotations let Spring's IoC container control object creation and wiring.

5. Step Four — Architectural Pattern

Roommate Dashboard implements the Model-View-Controller (MVC) pattern enforced by the Spring MVC framework, layered with a Repository pattern for data access and a Service layer for domain logic isolation.

5.1 MVC Layer Responsibilities

Layer	Technology	Classes	Responsibility
Model	JPA Entities + DTOs	AppUser, Household, Roommate, Expense, Chore, GroceryItem, Payment, HouseholdMember, ExpenseSplit, ActivityLog + all Form/DTO classes	Represent domain state; persisted via Hibernate; DTOs carry form data to/from templates
View	Thymeleaf Templates + CSS	dashboard.html, login.html, register.html, household-new.html, household-join.html, error.html, dashboard.css	Render HTML using Thymeleaf expressions bound to model attributes; no business logic in templates
Controller	Spring @Controller	DashboardController, HouseholdController, AuthController, ChoreController, ExpenseSplitController, GroceryController	Accept HTTP requests, invoke services, populate model, redirect or render view

5.2 Repository (Data Access) Layer

A dedicated repository package isolates all database access behind Spring Data JPA repository interfaces. Service classes call repository methods; they never construct JPQL or native SQL queries inline in business logic. This ensures the data access strategy can be changed (e.g., switching from H2 to PostgreSQL via the postgres profile) without altering service or controller code.

5.3 Service Layer (Domain Logic)

An intermediate service layer sits between controllers and repositories. This ensures:

- Controllers remain thin — they orchestrate calls but contain no calculation or rule logic
- Business rules (split calculation, rotating chores, balance netting) are co-located and testable in isolation
- Cross-cutting concerns (ActivityLogService, HouseholdContext) are injected rather than duplicated

5.4 Architectural Flow (Request Lifecycle)

Step	Actor	Action
1	Browser	HTTP POST /expenses/add with form fields
2	MockAuthInterceptor	Reads session; loads current user and household into request scope
3	ExpenseSplitController	Binds form to ExpenseForm DTO; calls expenseService.addExpense(form)
4	ExpenseService	Validates payer and participants; calculates splits; calls expenseRepo.save(), splitRepo.save(), activityLogService.log()
5	JPA/Hibernate	Executes INSERT statements; returns managed entities
6	ExpenseSplitController	Redirects to GET /dashboard
7	DashboardController	Calls DashboardModelBuilder to assemble model; returns 'dashboard' view name
8	Thymeleaf Engine	Renders dashboard.html with model data; sends HTML to browser

5.5 Database Configuration

Profile	Database	Purpose
Default (dev)	H2 in-memory or file (data/roommate_demo_db.mv.db)	Zero-config local development with demo seed data
postgres	PostgreSQL (configurable via environment variables)	Production deployment profile

6. Class Relationship Summary

From Class	Relationship	To Class	Multiplicity
Household	@ManyToOne	AppUser (createdBy)	Many Households → One AppUser
HouseholdMember	@ManyToOne	Household	Many Members → One Household
HouseholdMember	@ManyToOne	AppUser	Many Members → One AppUser
Roommate	@ManyToOne	Household	Many Roommates → One Household
Expense	@ManyToOne	Roommate (payer)	Many Expenses → One Roommate
Expense	@ManyToOne	Household	Many Expenses → One Household
ExpenseSplit	@ManyToOne	Expense	Many Splits → One Expense
ExpenseSplit	@ManyToOne	Roommate	Many Splits → One Roommate
Chore	@ManyToOne	Roommate (assignedTo)	Many Chores → One Roommate
Chore	@ManyToOne	Household	Many Chores → One Household
GroceryItem	@ManyToOne	Roommate (assignedTo)	Many Items → One Roommate
GroceryItem	@ManyToOne	Household	Many Items → One Household
Payment	@ManyToOne	Roommate (from)	Many Payments → One Roommate

From Class	Relationship	To Class	Multiplicity
Payment	@ManyToOne	Roommate (to)	Many Payments → One Roommate
Payment	@ManyToOne	Household	Many Payments → One Household
ActivityLog	@ManyToOne	Household	Many Logs → One Household
ActivityLog	@ManyToOne	AppUser (actor)	Many Logs → One AppUser

7. Test Coverage Summary

Test Class	Coverage Area	Tests Included
DashboardWorkflowTests	Integration — end-to-end domain workflows	addExpense_addsToList, markSplitPaid_updatesPaidStatus, addChore_assignedRoommate, completeChore_marksCompleted, addGroceryItem_appearsInList, markGroceryItemPurchased, logPayment_recordsTransfer, activityLog_capturesEvents, balance_reflectsExpensesAndPayments
RoomatedashApplicationTests	Spring Boot context load	contextLoads — verifies ApplicationContext starts without error

DashboardWorkflowTests uses @SpringBootTest with an H2 in-memory database and Mockito for session/auth mocking. Tests call service methods directly to verify domain logic, then assert repository state after the operation.

8. Deliverable Checklist

Requirement	Status	Where to Find It
Step 1 — Identify analysis classes using the IAC method	Complete	Section 2 — Entity, Control, Boundary, DTO classes identified and documented with stereotypes
Step 2 — Abstraction	Complete	Section 3.2 — JpaRepository interfaces, service abstraction, DashboardModelBuilder

Requirement	Status	Where to Find It
Step 2 — Inheritance	Complete	Section 3.3 — DashboardException/ResourceNotFoundException hierarchy; JpaRepository inheritance
Step 2 — Polymorphism	Complete	Section 3.4 — Interface polymorphism (repos), exception handler dispatch, enum-driven rendering
Step 2 — Encapsulation	Complete	Section 3.1 — Private fields, public accessors, domain methods (complete(), markPaid(), markPurchased())
Step 2 — Meaningful comments	Complete	Service and entity classes include inline comments; reviewable in src/main/java/
Step 3 — SOLID principles	Complete	Section 4 — All five principles addressed with concrete class references
Step 4 — Architectural pattern (SRS-aligned)	Complete	Section 5 — MVC + Repository + Service layering; matches Phase 1 architectural decision
Document summarizing where everything is	Complete	This document — Section 1 maps all packages, files, and locations
Organized code package	Complete	src/ directory structured by package (model, service, controller, repo, dto, config, exception)
Supporting artifacts	Complete	Test classes, Thymeleaf templates, CSS, application.properties, pom.xml included in ZIP